Universidad Camilo José Cela
SEK EDUCATION GROUP

ADMINISTRACIÓN Y SERVICIOS DE SISTEMAS OPERATIVOS

88020

PRÁCTICA 2

SISTEMA DE ARCHIVOS, COMANDOS, USUARIOS Y PERMISOS

Alumnos: Eva María Ramos Juárez
Alberto García Martín
Javier Jiahao Cheng Hu
Grado: Inteligencia Artificial y Robótica
Profesor: Domingo Sampedro Lirio

Identificador del documento: 88020_P2_AGJCER

ÍNDICE DE CONTENIDO

1. Introducción..... 7
2. Desarrollo..... 9
2.1. Identificación del usuario y exploración del sistema..... 9
2.2. Estructura de directorios del proyecto..... 9
2.3. Usuarios, grupos y cuentas de los responsables..... 10
2.4. Permisos DAC sobre el directorio de documentación..... 11
2.5. Política de permisos por roles con DAC y ACL..... 12
2.6. Validación individual de permisos..... 14
2.7. Mejoras dentro de la política de permisos..... 17
2.8. Cuenta de dirección y separación de la administración..... 19
2.9. Acceso remoto de los responsables por SSH..... 20
3. Problemas encontrados y análisis de soluciones propuestas..... 23
4. Reparto de tareas (trabajos grupales)..... 27
5. Conclusiones..... 29

ÍNDICE DE TABLAS

Tabla 1. Usuarios del grupo y grupos a los que pertenecen..... 11
Tabla 2. Resumen de la política de permisos aplicada..... 14
Tabla 3. Validación del miembro A (eva)..... 15
Tabla 4. Validación del miembro B (alberto)..... 16
Tabla 5. Validación del miembro C (javier)..... 16
Tabla 6. Resumen de los problemas encontrados y de su solución..... 25

ÍNDICE DE ILUSTRACIONES

Ilustración 1. Pantalla de inicio del script en el servidor server-asio-01..... 7
Ilustración 2. Identificación con whoami, hostname y pwd y exploración de / y /home..... 9
Ilustración 3. Acceso restringido a /root para eva y acceso mediante sudo desde la cuenta director..... 9
Ilustración 4. Creación de la estructura de /srv/aurora6 y comprobación con tree -d..... 10
Ilustración 5. Creación de los grupos de departamento y del grupo común proyecto..... 10
Ilustración 6. Cuentas de los responsables con contraseña temporal y cambio obligatorio..... 11
Ilustración 7. Comprobación de pertenencias con id y groups..... 11
Ilustración 8. Asignación del grupo proyecto y de los permisos 751 al directorio documentacion..... 12
Ilustración 9. Base común: raíz del proyecto, ingeniería, intercambio y confidencial..... 12
Ilustración 10. Miembro A: grupo mecanica, permisos 2770 y ACL de lectura para avionica..... 13
Ilustración 11. Miembro B: grupos avionica y software y ACL de lectura para mecanica..... 13
Ilustración 12. Miembro C: compras, documentacion/comun y ACL de lectura sobre las facturas..... 13
Ilustración 13. Accesos directos de cada responsable, ficheros de ejemplo y resumen de permisos..... 14
Ilustración 14. Validación del miembro A (eva)..... 15
Ilustración 15. Validación del miembro B (alberto)..... 15
Ilustración 16. Validación del miembro C (javier)..... 16
Ilustración 17. getfacl ejecutado por cada responsable sobre su área..... 17
Ilustración 18. Matriz de accesos comprobada en el servidor tras las correcciones..... 17
Ilustración 19. Permisos del flujo incoming, approved y quarantine..... 18
Ilustración 20. Prueba del sticky bit y del flujo de aprobación..... 18
Ilustración 21. Copia de permisos y ruptura intencionada de dos permisos..... 18
Ilustración 22. Restauración de los permisos con setfacl --restore..... 18
Ilustración 23. Plantilla /etc/skel con el fichero de bienvenida..... 18
Ilustración 24. Política de contraseñas y creación de la cuenta temporal becario..... 19
Ilustración 25. Validación del becario: solo lectura en documentacion/comun..... 19
Ilustración 26. Informe de auditoría de /srv/aurora6..... 19
Ilustración 27. Cuenta director: grupos, acceso directo y único miembro del grupo sudo..... 20
Ilustración 28. eva ya no pertenece al grupo sudo y no puede usar privilegios de administración..... 20
Ilustración 29. director solo accede a confidencial mediante sudo..... 20
Ilustración 30. Matriz de accesos final con las cuentas becario y director..... 20
Ilustración 31. Usuario alberto (miembro B) conectado por SSH desde otro ordenador: escritura permitida en avionica y denegada en mecanica..... 21
Ilustración 32. Usuario javier (miembro C) conectado por SSH desde otro ordenador: escritura permitida en compras/pedidos y denegada en mecanica..... 21
Ilustración 33. Corrección: tras añadir las ACL por defecto, los ficheros nuevos son editables por el grupo..... 23

Ilustración 34. Primera matriz de accesos, incorrecta por la incompatibilidad de /usr/bin/test.....23
Ilustración 35. Regla de reenvío limitada a 127.0.0.1, corrección con la IP del anfitrión vacía y conexión SSH por la dirección del Mac en la red.....25
Ilustración 36. VBoxManage ejecutado por error dentro del servidor: el comando solo existe en el Mac anfitrión.....25

1. Introducción

En esta segunda práctica se prepara el servidor Ubuntu instalado en la práctica 1 para que lo utilicen varias personas de la empresa ficticia **AURORA Aerospace**, que desarrolla el proyecto **AURORA-6**. El objetivo es organizar la información del proyecto en un árbol de directorios bajo `/srv/aurora6`, crear los usuarios y grupos de cada departamento y aplicar una política de permisos en la que cada persona trabaje en su área, consulte lo que necesita de otras áreas y no pueda modificar ni ver lo que no le corresponde.

Para ello se combinan los permisos clásicos de Linux, es decir, el **control de acceso discrecional (DAC)** con propietario, grupo y otros, con las **listas de control de acceso (ACL)**, que permiten dar permisos concretos a grupos adicionales sin cambiar el propietario ni el grupo de un directorio. Además, se ha seguido el principio de **mínimo privilegio**: cada usuario recibe solo los permisos que necesita y las tareas de administración se hacen siempre con `sudo`.

En la práctica intervienen tres roles. El miembro A (usuario `eva`) es responsable de Ingeniería mecánica, el miembro B (usuario `alberto`) de Aviónica y Software, y el miembro C (usuario `javier`) de Compras y configuración. Cada responsable tiene su propia cuenta con contraseña y un acceso directo al área que controla. Por otro lado, la administración del servidor no recae en ninguno de ellos, sino en una cuenta de dirección (`director`), que es la única con privilegios de administración; el usuario `eva` queda así como responsable de Mecánica sin `sudo`. La persona que realizó cada parte se indica en el apartado 4.

Todos los comandos se han reunido en el script `88020_P2_Script_AGJCER.sh`, que muestra cada orden con el usuario que la ejecuta, indica en verde o en rojo si el sistema la ha permitido o denegado, sustituye los mensajes de error del sistema por avisos claros como «Acceso restringido: no dispones de los permisos necesarios» y se detiene entre apartados para poder revisarlos. El script se ha ejecutado en el servidor `server-asio-01`, con Ubuntu Server 26.04.1 LTS sobre VirtualBox.



Ilustración 1. *Pantalla de inicio del script en el servidor server-asio-01.*

2. Desarrollo

2.1. Identificación del usuario y exploración del sistema

En primer lugar, se ha comprobado quién es el usuario y dónde se encuentra con `whoami`, que muestra el nombre del usuario actual (`eva`), `hostname`, que muestra el nombre del equipo (`server-asio-01`), y `pwd`, que indica el directorio de trabajo, en este caso el directorio personal `/home/eva`. A continuación, con `ls /` se ha revisado la raíz del sistema de archivos, donde aparecen los directorios estándar como `/etc` (configuración), `/home` (datos de los usuarios), `/srv` (datos de los servicios, donde irá el proyecto) o `/var` (datos variables y registros). Por otra parte, `ls -l /home` muestra que el directorio personal de `eva` tiene permisos `drwxr-x---`, de modo que el resto de usuarios no puede entrar en él.

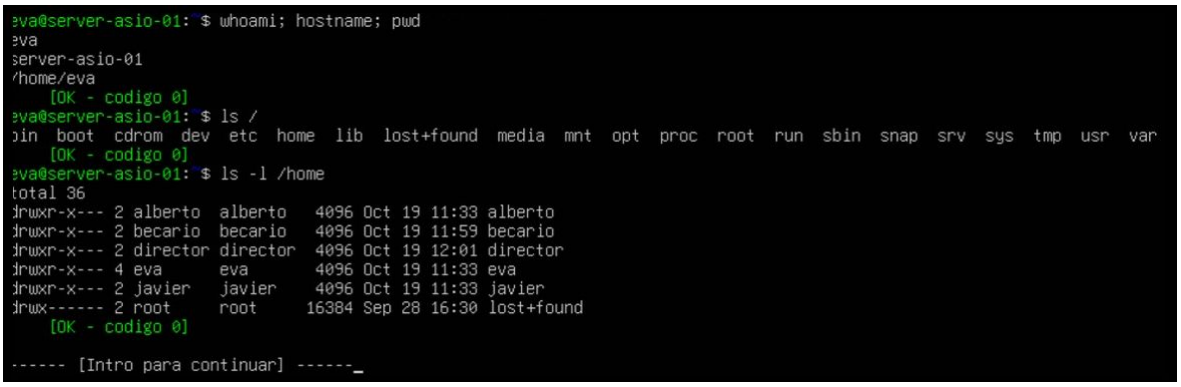


Ilustración 2. *Identificación con whoami, hostname y pwd y exploración de / y /home.*

Sin embargo, al ejecutar `ls /root` con un usuario normal el acceso se deniega y el script muestra «Acceso restringido: no dispones de los permisos necesarios», ya que `/root` es el directorio personal del administrador y tiene permisos `drwx------`, es decir, solo `root` puede leerlo. En cambio, la cuenta administradora `director` sí puede mostrar su contenido con `sudo`, porque `sudo` ejecuta ese comando concreto con privilegios de administrador después de pedir su contraseña.



Ilustración 3. *Acceso restringido a /root para eva y acceso mediante sudo desde la cuenta director.*

No se trabaja directamente como `root` porque cualquier error tendría efecto inmediato sobre todo el sistema y, además, no quedaría claro quién ha hecho cada cambio. Por el contrario, con `sudo` cada acción de administración es explícita, se limita a un único comando y queda registrada en `/var/log/auth.log` con el nombre del usuario que la ejecutó. Por este motivo, en Ubuntu la cuenta `root` viene bloqueada por defecto y el usuario creado en la instalación pertenece al grupo `sudo`.

2.2. Estructura de directorios del proyecto

La estructura del proyecto se ha creado bajo `/srv/aurora6`, ya que `/srv` es el directorio que el estándar de jerarquía del sistema de archivos (FHS) reserva para los datos que sirve el equipo. Para crearla en un solo paso se ha usado `mkdir -p` con expansión de llaves, que crea también los directorios intermedios y no da

error si alguno ya existe. De este modo, el árbol queda formado por `ingenieria` (con `mecanica`, `avionica` y `software`), `compras` (con `proveedores` y `pedidos`), `contabilidad/facturas`, `documentacion/comun`, `confidencial` e `intercambio` (con `incoming`, `approved` y `quarantine`). Finalmente, `tree -d` dibuja solo los directorios y confirma que existen los 17 esperados.



Ilustración 4. *Creación de la estructura de /srv/aurora6 y comprobación con tree -d.*

2.3. Usuarios, grupos y cuentas de los responsables

Los grupos se han diseñado a partir de la organización de la empresa. Por un lado, se ha creado un grupo por cada departamento que tiene un área de trabajo propia: `mecanica` para Ingeniería mecánica, `avionica` para Aviónica, `software` para Software y `compras` para Compras. Aviónica y Software se han separado en dos grupos aunque los lleve el mismo responsable, porque son áreas distintas con directorios distintos y así, si en el futuro se incorpora una persona solo a Software, bastaría con añadirla a ese grupo. Por otro lado, se ha creado el grupo `proyecto`, que reúne a todos los miembros y se utiliza para los recursos comunes, como la raíz `/srv/aurora6`, la documentación y el intercambio.

Para crearlos se ha utilizado `groupadd` dentro de un bucle `for` que recorre los cinco nombres. Antes de crear cada grupo, el bucle comprueba con `getent group` si ya existe, de forma que el script se puede repetir sin errores. El sistema asigna a cada grupo un identificador numérico (GID) consecutivo, y la salida de `getent group` muestra que se han creado `mecanica` (1001), `avionica` (1002), `software` (1003), `compras` (1004) y `proyecto` (1005), todavía sin miembros.

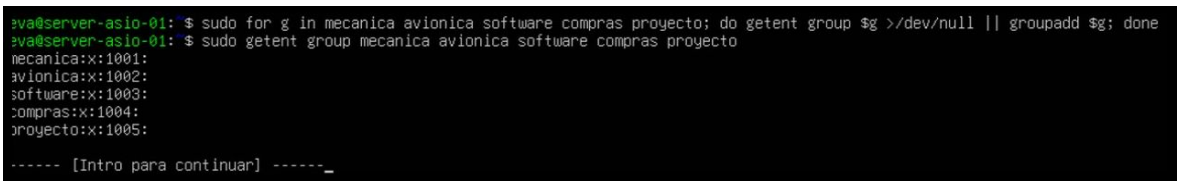


Ilustración 5. *Creación de los grupos de departamento y del grupo común proyecto.*

Para que cada responsable pueda entrar en el servidor con su propio usuario y contraseña, las cuentas `alberto` y `javier` se han creado con `useradd -m -s /bin/bash`, que genera su directorio personal y les asigna el intérprete Bash. En lugar de inventar una contraseña fija, el script genera una contraseña temporal aleatoria con `openssl rand`, la asigna con `chpasswd` y la guarda en `/root/aurora6_credenciales.txt` con permisos `600`, de modo que solo el administrador puede leerla para entregársela a cada responsable. Además, `chage -d 0` marca la contraseña como caducada, por lo que el sistema obliga a cambiarla en el primer inicio de sesión. La salida de `passwd -S` lo confirma: la `P` indica que la cuenta tiene contraseña y la fecha `1970-01-01` indica que debe cambiarse. La cuenta `eva` ya existía desde la instalación y conserva su contraseña.

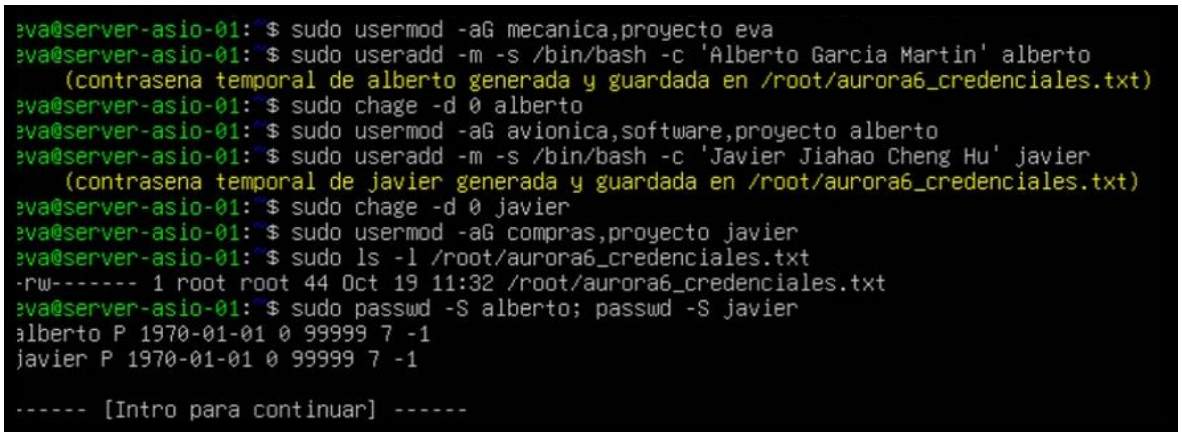


Ilustración 6. *Cuentas de los responsables con contraseña temporal y cambio obligatorio.*

Después, con `usermod -aG` se ha añadido cada usuario a los grupos que corresponden a su rol. El usuario `eva` (miembro A) se ha añadido a `mecanica` y `proyecto`, ya que solo trabaja en Ingeniería mecánica y en lo común. El usuario `alberto` (miembro B) se ha añadido a `avionica`, `software` y `proyecto`, porque es responsable de las dos áreas. Por último, el usuario `javier` (miembro C) se ha añadido a `compras` y `proyecto`. La opción `-a` es imprescindible, ya que sin ella `usermod -G` sustituiría los grupos secundarios existentes en lugar de añadir los nuevos; en el caso de `eva`, por ejemplo, perdería su pertenencia a `sudo` y al resto de grupos creados en la instalación. Además, cada usuario conserva su grupo privado (con su mismo nombre), que es su grupo principal y el que se asigna por defecto a sus ficheros personales.

Finalmente, se ha comprobado la pertenencia con `id`, que muestra el UID, el grupo principal y los grupos secundarios de cada usuario, y con `groups`, que los lista de forma más sencilla. En la salida se ve que cada miembro pertenece exactamente a los grupos de su rol, lo que es la base de toda la política de permisos de los apartados siguientes.

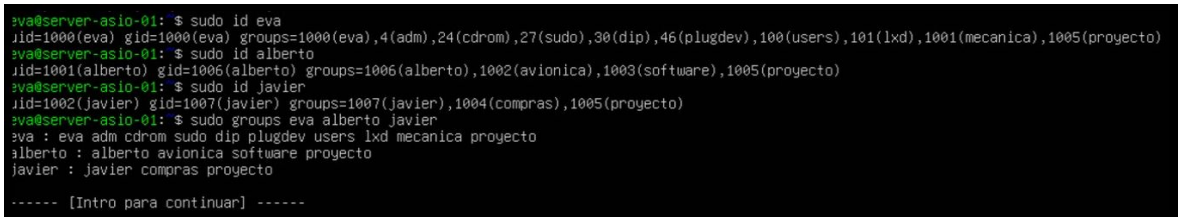


Ilustración 7. *Comprobación de pertenencias con id y groups.*

Miembro	Usuario	Rol	Grupos secundarios
A	eva	Ingeniería mecánica	mecanica, proyecto
B	alberto	Aviónica y Software	avionica, software, proyecto
C	javier	Compras y configuración	compras, proyecto

Tabla 1. *Usuarios del grupo y grupos a los que pertenecen.*

2.4. Permisos DAC sobre el directorio de documentación

En este apartado se ha asignado el directorio `documentacion` al grupo `proyecto` con `chown root:proyecto` y se le han aplicado los permisos `751` con `chmod`. Como resultado, `ls -ld` y `stat` muestran `drwxr-x--x 751 root:proyecto`.



Ilustración 8. *Asignación del grupo proyecto y de los permisos 751 al directorio documentacion.*

Cada cifra del modo octal corresponde a una clase de usuarios y es la suma de lectura (4), escritura (2) y ejecución (1). Por lo tanto, el 7 da al propietario (`root`) permisos `rwx`, el 5 da al grupo `proyecto` permisos `r-x` y el 1 da al resto de usuarios solo `--x`. En un directorio, el permiso de lectura permite listar su contenido, el de escritura permite crear, renombrar y borrar entradas, y el de ejecución permite atravesarlo, es decir, entrar en él o acceder a un fichero cuyo nombre se conoce. En consecuencia, con `751` los miembros del proyecto pueden listar y recorrer la documentación pero no modificarla, mientras que los demás usuarios solo pueden atravesar el directorio si conocen la ruta exacta, sin poder ver qué contiene.

No obstante, al aplicar la política completa del apartado siguiente este permiso se ha endurecido a `750`, ya que todos los usuarios que deben acceder a la documentación pertenecen a `proyecto` y no hay ningún motivo para que otros usuarios del sistema puedan atravesar el directorio.

2.5. Política de permisos por roles con DAC y ACL

Antes de aplicar los permisos de cada rol se ha preparado una base común. Por un lado, `/srv/aurora6` e `ingenieria` pertenecen a `root:proyecto` con permisos `750`, de modo que solo los miembros del proyecto pueden entrar en el árbol. Por otro lado, `intercambio` y sus subdirectorios tienen `2770` con el grupo `proyecto`, y `confidencial` pertenece a `root:root` con `700`, por lo que solo se puede acceder a él con `sudo`.



Ilustración 9. *Base común: raíz del proyecto, ingeniería, intercambio y confidencial.*

En los directorios de trabajo se ha usado el permiso especial **setgid** (el 2 inicial de `2770`), que en `ls -l` aparece como una `s` en el lugar de la ejecución del grupo. Este bit hace que los ficheros creados dentro hereden el grupo del directorio en lugar del grupo privado del usuario, lo que garantiza que el resto del departamento pueda trabajar con ellos. Asimismo, las ACL se han definido también como ACL por defecto (`d:`), que se copian automáticamente a los ficheros y subdirectorios nuevos y evitan tener que repetir `setfacl` cada vez que alguien crea un documento.

Para el miembro A, el directorio `ingenieria/mecanica` se ha configurado con el grupo `mecanica` y permisos `2770`, de modo que solo su departamento puede escribir. Además, se ha añadido una ACL `g:avionica:rx` para que Aviónica pueda consultar los diseños mecánicos sin modificarlos, ya que ambos equipos trabajan sobre el mismo vehículo. En la salida de `getfacl` se aprecia la entrada `group:avionica:r-x` y la marca `flags: -s-`, que corresponde al bit setgid.



Ilustración 10. *Miembro A: grupo mecanica, permisos 2770 y ACL de lectura para avionica.*

En el caso del miembro B, `ingenieria/avionica` e `ingenieria/software` tienen sus grupos respectivos y permisos `2770`, y en `avionica` se ha añadido una ACL `g:mecanica:rx` para que Mecánica pueda consultar la información de aviónica. La máscara (`mask::rwx`) no recorta los permisos de la entrada añadida, por lo que el permiso efectivo de `mecanica` es exactamente `r-x`.



Ilustración 11. *Miembro B: grupos avionica y software y ACL de lectura para mecanica.*

Para el miembro C, `compras` y sus subdirectorios tienen el grupo `compras` y permisos `2770`, mientras que `documentacion/comun` tiene con `root:proyecto`, permisos `2750` y una ACL `g:compras:rwx`, de manera que todo el proyecto puede leer la documentación común pero solo Compras, encargado de la configuración, puede mantenerla. En cuanto a las facturas, `contabilidad` pertenece a `root` con `700` y una ACL `g:compras:x`, que solo permite atravesarlo, mientras que `facturas` tiene una ACL `g:compras:rx`. De esta forma, Compras puede leer las facturas pero no modificarlas ni ver el resto de contabilidad.

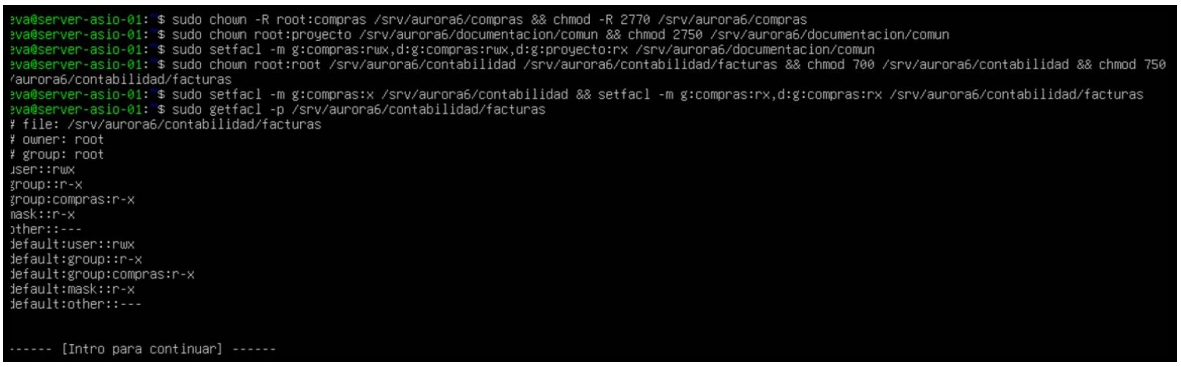


Ilustración 12. *Miembro C: compras, documentacion/comun y ACL de lectura sobre las facturas.*

Para que cada responsable llegue directamente a lo que controla, el script crea en su directorio personal un enlace simbólico con `ln -sfn`: el usuario `eva` tiene `~/mecanica`, `alberto` tiene `~/avionica` y `~/software`, y `javier` tiene `~/compras`. El enlace no concede ningún permiso por sí mismo, ya que el acceso sigue dependiendo de los permisos del directorio de destino. Finalmente, se han creado tres ficheros de ejemplo (una factura, las normas comunes y un plan confidencial) y se ha revisado el resultado con `ls -l`, donde el signo `+` al final de los permisos indica qué directorios tienen ACL.



Ilustración 13. *Accesos directos de cada responsable, ficheros de ejemplo y resumen de permisos.*

Se han utilizado ACL en lugar de cambiar el propietario, el grupo o los permisos de otros porque el modelo DAC solo admite un grupo por fichero o directorio. Si se cambiara el grupo de `mecanica` a `avionica`, Mecánica perdería el control de su propio directorio; si se dieran permisos a otros, cualquier usuario del sistema podría leer los diseños; y crear un grupo nuevo que mezclara ambos departamentos complicaría la gestión cada vez que cambiara una persona. Por consiguiente, la ACL es la solución más precisa, ya que añade un permiso concreto a un grupo concreto y mantiene intacto el resto de la configuración.

Directorio	Propietario:grupo	Modo	ACL añadidas
/srv/aurora6, ingenieria	root:proyecto	750	—
ingenieria/mecanica	root:mecanica	2770	avionica r-x
ingenieria/avionica	root:avionica	2770	mecanica r-x
ingenieria/software	root:software	2770	por defecto software rwx
compras (y subdirectorios)	root:compras	2770	por defecto compras rwx
documentacion	root:proyecto	750	—
documentacion/comun	root:proyecto	2750	compras rwx
contabilidad	root:root	700	compras --x
contabilidad/facturas	root:root	750	compras r-x
intercambio (y subdirectorios)	root:proyecto	2770	—
confidencial	root:root	700	— (solo sudo)

Tabla 2. *Resumen de la política de permisos aplicada.*

2.6. Validación individual de permisos

Para validar la política, cada usuario ha ejecutado sus pruebas con su propia cuenta, sin `sudo`, y el script indica en verde `[OK]` las operaciones permitidas y en rojo `[DENEGADO]`, junto al aviso «Acceso restringido: no dispones de los permisos necesarios», las que el sistema ha rechazado. En todos los casos el resultado obtenido coincide con el esperado.

El usuario `eva` (miembro A) ha creado un fichero en `mecanica` a través de su acceso directo, y en el listado se observa que hereda el grupo `mecanica` gracias al bit setgid y que lleva el signo `+` de la ACL por defecto.

Además, ha podido listar `avionica` gracias a la ACL y leer las normas comunes del proyecto. En cambio, no ha podido crear ficheros en `avionica` ni entrar en `contabilidad` o `confidencial`.



Ilustración 14. *Validación del miembro A (eva).*

Usuario	Comando	Esperado	Obtenido
eva	echo … > mecanica/ensayo_estructura.txt	Permitido	OK (código 0)
eva	ls ingenieria/avionica; cat documentacion/comun/normas_proyecto.txt	Permitido (lectura)	OK (código 0)
eva	touch ingenieria/avionica/prueba_eva.txt	Denegado	Acceso restringido
eva	ls contabilidad	Denegado	Acceso restringido
eva	ls confidencial	Denegado	Acceso restringido

Tabla 3. *Validación del miembro A (eva).*

Por su parte, el usuario `alberto` (miembro B) ha escrito en `avionica` y en `software` y ha leído el fichero de ensayo creado por `eva` en `mecanica`. Sin embargo, no ha podido escribir en `mecanica`, ya que su ACL es solo de lectura, ni acceder a `compras` o `confidencial`.

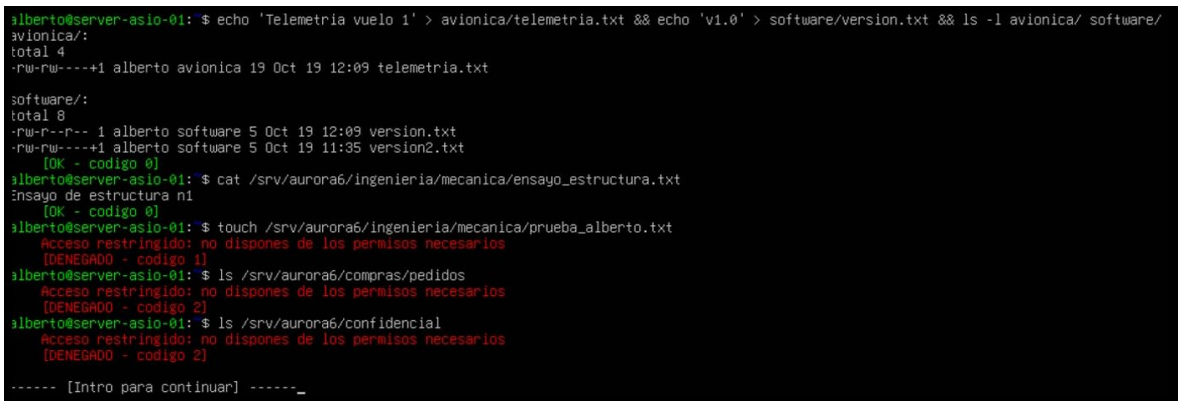


Ilustración 15. *Validación del miembro B (alberto).*

Usuario	Comando	Esperado	Obtenido
alberto	echo … > avionica/telemetria.txt; echo … > software/version.txt	Permitido	OK (código 0)
alberto	cat ingenieria/mecanica/ensayo_estructura.txt	Permitido (lectura)	OK (código 0)
alberto	touch ingenieria/mecanica/prueba_alberto.txt	Denegado	Acceso restringido
alberto	ls compras/pedidos	Denegado	Acceso restringido
alberto	ls confidencial	Denegado	Acceso restringido

Tabla 4. Validación del miembro B (alberto).

Finalmente, el usuario `javier` (miembro C) ha registrado un pedido en `compras/pedidos`, ha leído una factura y ha listado la documentación común. No obstante, no ha podido crear ficheros en `facturas`, lo que confirma que su acceso a contabilidad es únicamente de lectura, ni escribir en `mecanica` ni entrar en `confidencial`.

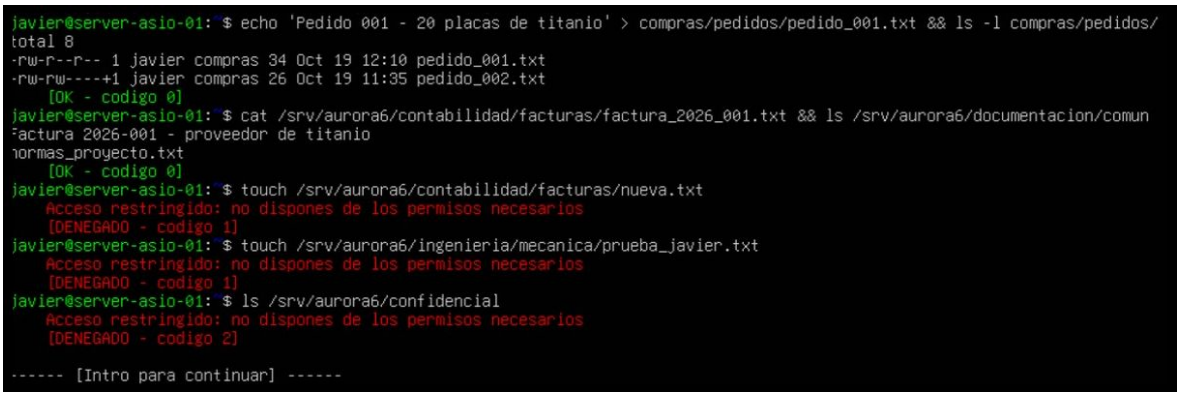


Ilustración 16. Validación del miembro C (javier).

Usuario	Comando	Esperado	Obtenido
javier	echo … > compras/pedidos/pedido_001.txt	Permitido	OK (código 0)
javier	cat contabilidad/facturas/factura_2026_001.txt; ls documentacion/comun	Permitido (lectura)	OK (código 0)
javier	touch contabilidad/facturas/nueva.txt	Denegado	Acceso restringido
javier	touch ingenieria/mecanica/prueba_javier.txt	Denegado	Acceso restringido
javier	ls confidencial	Denegado	Acceso restringido

Tabla 5. Validación del miembro C (javier).

Asimismo, cada responsable ha consultado con `getfacl` la ACL de su área sin necesidad de `sudo`, lo que demuestra que la lectura de los permisos está al alcance de quien gestiona cada directorio.

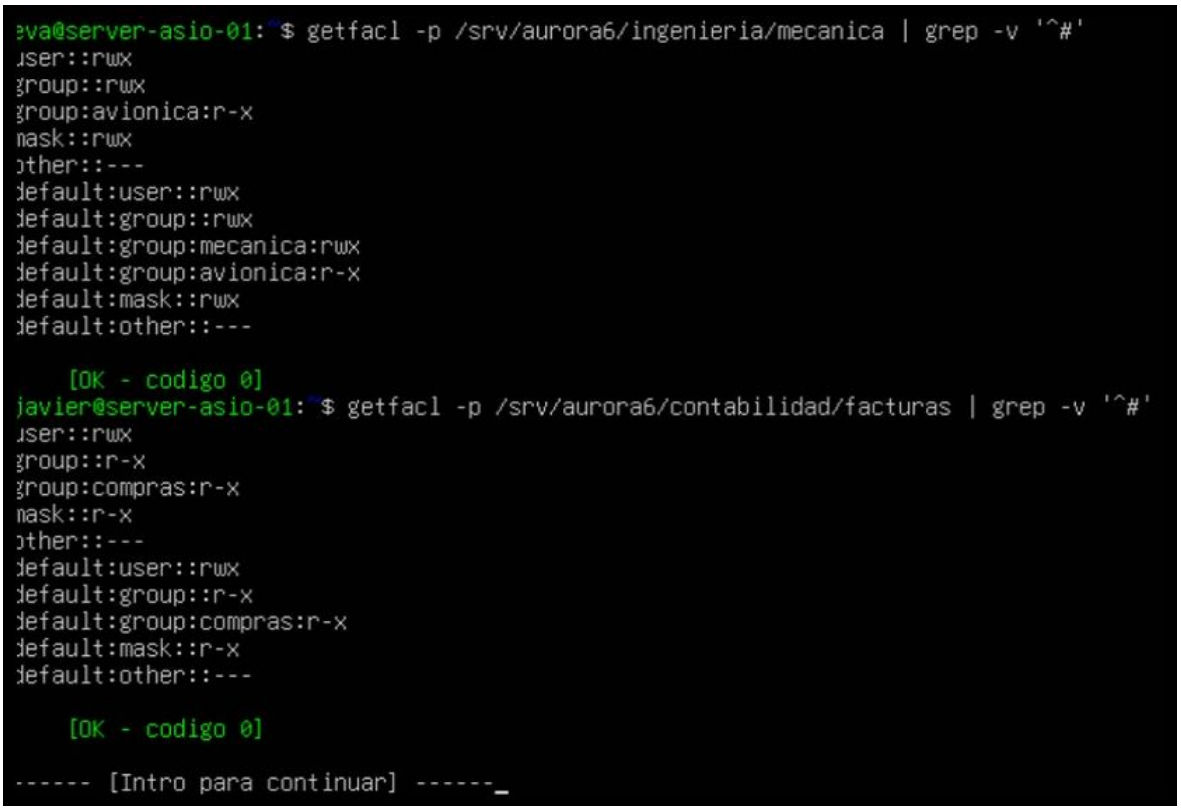


Ilustración 17. *getfacl ejecutado por cada responsable sobre su área.*

Para terminar, el script genera una **matriz de accesos** que comprueba en el propio sistema, para cada usuario y cada directorio, si puede leer (L) y escribir (E). La matriz resume en una sola pantalla toda la política: cada responsable escribe solo en su área, lee lo que comparte con otros departamentos y nadie accede a `confidencial` sin `sudo`.

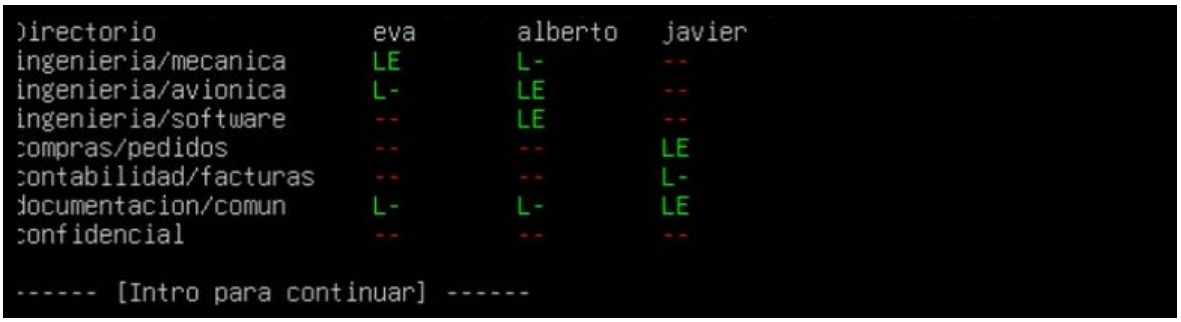


Ilustración 18. *Matriz de accesos comprobada en el servidor tras las correcciones.*

2.7. Mejoras dentro de la política de permisos

Además de lo anterior, se han añadido varias mejoras que no introducen temas nuevos, sino que aprovechan los mismos permisos, usuarios y comandos de la práctica. En primer lugar, el directorio `intercambio` se ha convertido en un flujo de trabajo. En `incoming` cualquier miembro del proyecto puede dejar ficheros, y se ha activado el **sticky bit** (el 1 inicial de `3770`, que aparece como una `T` en `ls -l`), de modo que cada usuario solo puede borrar sus propios ficheros. Por su parte, `approved` lo puede leer todo el proyecto pero solo Compras puede añadir documentos aprobados, y `quarantine` queda reservado a Compras.

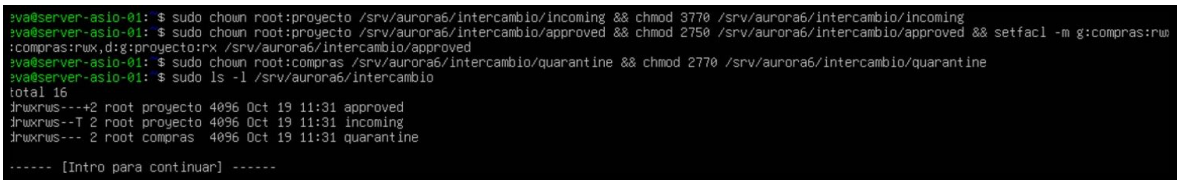


Ilustración 19. *Permisos del flujo incoming, approved y quarantine.*

En la prueba, el usuario `eva` deja un plano en `incoming` y `alberto` no puede borrarlo aunque tenga escritura en el directorio, precisamente por el sticky bit. En cambio, `javier` lo copia a `approved`, mientras que `alberto` no puede añadir ficheros ahí ni ver el contenido de `quarantine`.



Ilustración 20. *Prueba del sticky bit y del flujo de aprobación.*

En segundo lugar, se ha guardado una **copia de seguridad de todos los permisos y ACL** del árbol con `getfacl -R`. Para comprobar que sirve, se rompieron a propósito dos permisos, abriendo `confidencial` a todos con `777` y eliminando la ACL de `mecanica` con `setfacl -b`, y después se restauraron con `setfacl --restore`, que devolvió exactamente los permisos, propietarios y ACL originales.

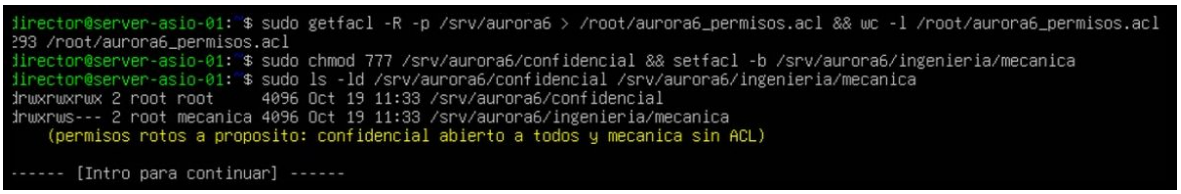


Ilustración 21. *Copia de permisos y ruptura intencionada de dos permisos.*



Ilustración 22. *Restauración de los permisos con setfacl --restore.*

Asimismo, se ha añadido un fichero de bienvenida `LEEME_AURORA6.txt` en `/etc/skel`, la plantilla que `useradd -m` copia en el directorio personal de cada cuenta nueva, de modo que todos los usuarios que se creen a partir de ahora lo reciben automáticamente.



Ilustración 23. *Plantilla /etc/skel con el fichero de bienvenida.*

Por otra parte, se ha aplicado una **política de contraseñas** con `chage`, que obliga a cambiarlas como máximo cada 90 días, impide cambiarlas más de una vez al día y avisa siete días antes de que caduquen.

Además, se ha creado una cuenta `becario` con fecha de caducidad el 31 de diciembre de 2026, pensada para una persona en prácticas que solo debe consultar la documentación común. A diferencia de los departamentos, su acceso se ha dado con **ACL de usuario** (`u:becario`), que permiten atravesar `/srv/aurora6` y `documentacion` y leer `documentacion/comun`, sin añadirlo a ningún grupo. En la validación, el becario lee las normas del proyecto, pero no puede escribir ni listar el resto del árbol.

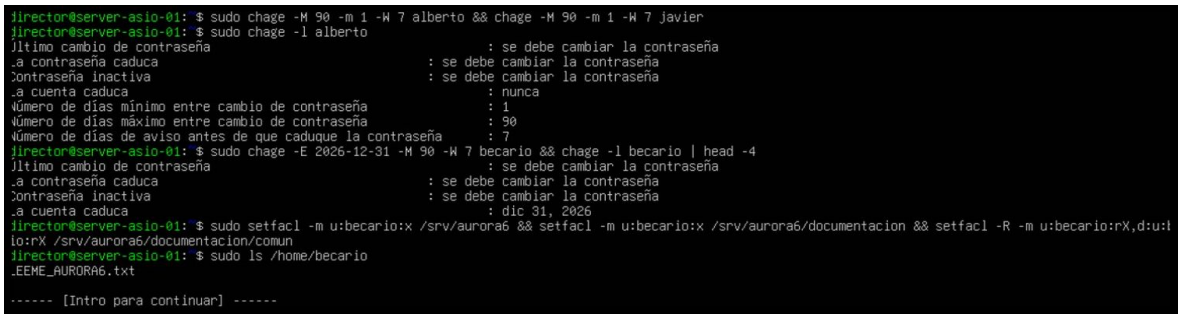


Ilustración 24. *Política de contraseñas y creación de la cuenta temporal becario.*

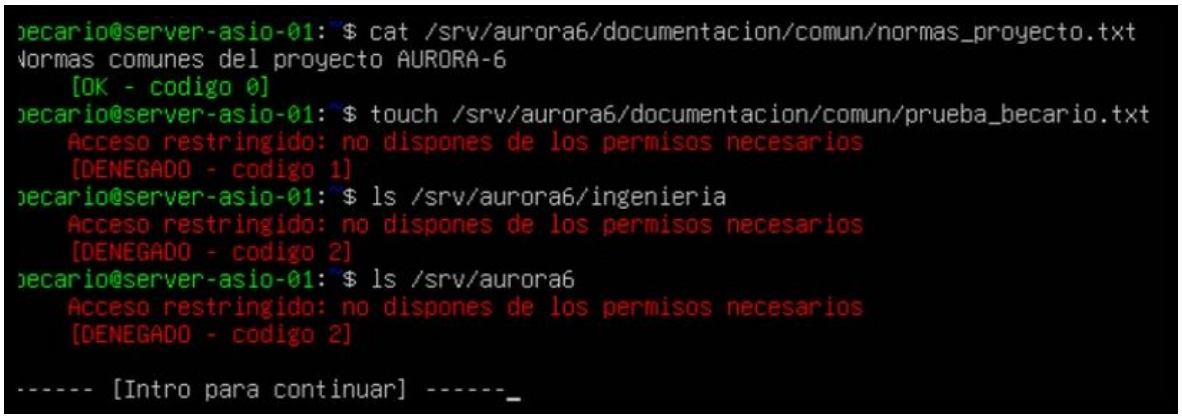


Ilustración 25. *Validación del becario: solo lectura en documentacion/comun.*

Finalmente, el script genera un **informe de auditoría** que busca directorios escribibles por cualquier usuario y ficheros sin propietario ni grupo válido, cuenta los directorios con setgid y los elementos con ACL y muestra el espacio que ocupa cada área con `du`. El informe se guarda en `/root/auditoria_aurora6.txt` y confirma que no existe ningún permiso peligroso en el árbol.

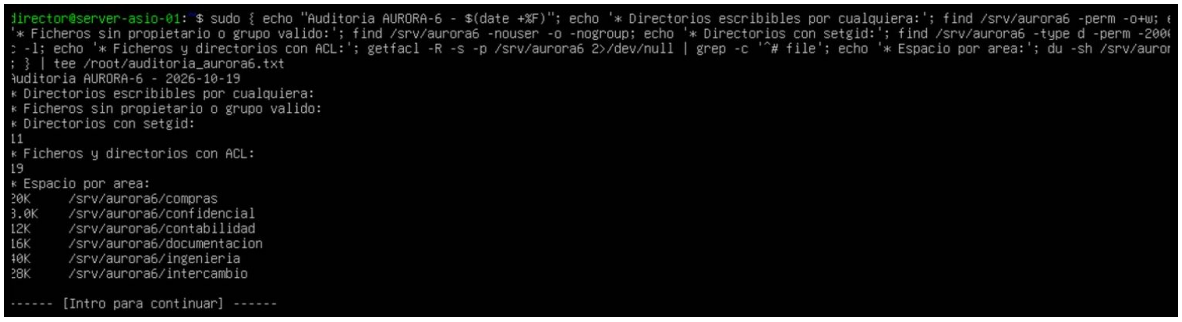


Ilustración 26. *Informe de auditoría de /srv/aurora6.*

2.8. Cuenta de dirección y separación de la administración

Hasta este punto, el usuario `eva` desempeñaba dos papeles a la vez, ya que era responsable de Mecánica y también administrador del servidor. Se ha creado una cuenta aparte, `director`, que representa a la dirección del proyecto y no está vinculada a ningún miembro del grupo. Se ha optado por un usuario nuevo en lugar de mantener la administración en una de las cuentas de los responsables por varios motivos. En primer lugar, así se aplica el principio de mínimo privilegio también a la administración: cada responsable

trabaja solo con los permisos de su área y nadie tiene privilegios de administrador por el hecho de dirigir un departamento. En segundo lugar, se separan las funciones, ya que las tareas de administración quedan registradas a nombre de `director` y no se mezclan con el trabajo diario de Mecánica. Por último, si en el futuro cambia el responsable de un departamento, basta con modificar su cuenta sin tocar la administración del servidor, y viceversa. Su contraseña se eligió en el momento de crearla y no queda guardada en ningún fichero. Esta cuenta pertenece al grupo `sudo` y a todos los grupos de departamento, por lo que puede trabajar en todas las áreas, y tiene un acceso directo `~/aurora6` a todo el proyecto.

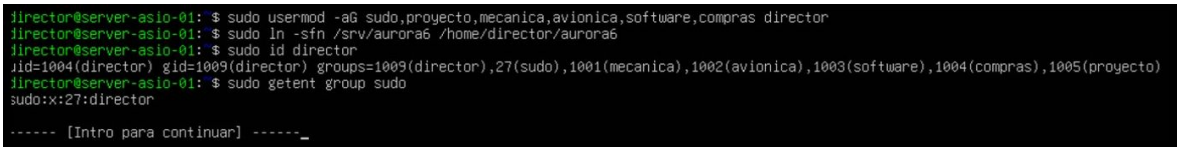


Ilustración 27. *Cuenta director: grupos, acceso directo y único miembro del grupo sudo.*

A continuación, con `gpasswd -d eva sudo` se ha retirado a `eva` del grupo `sudo`, de modo que queda únicamente como responsable de Mecánica. Cuando intenta usar `sudo` para entrar en `confidencial`, el script muestra «Acceso restringido: no tienes permisos de administrador». En cambio, `director` no puede listar `confidencial` como usuario normal, pero sí con `sudo`, de modo que la información confidencial solo es accesible con privilegios de administración.

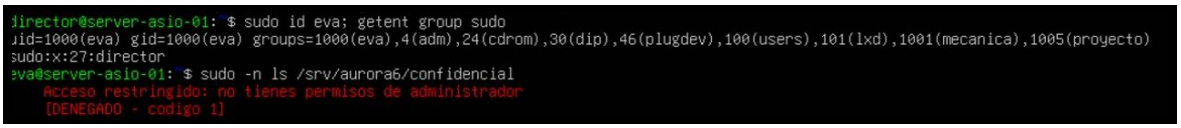


Ilustración 28. *eva ya no pertenece al grupo sudo y no puede usar privilegios de administración.*

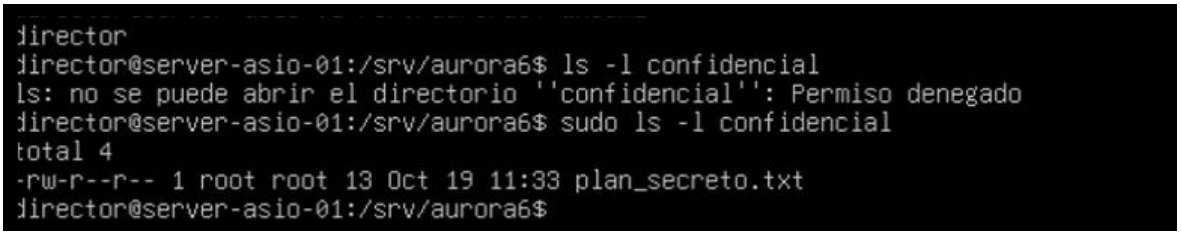


Ilustración 29. *director solo accede a confidencial mediante sudo.*

La matriz de accesos final, ejecutada ya desde la cuenta `director`, resume la política completa con las cinco cuentas del servidor.

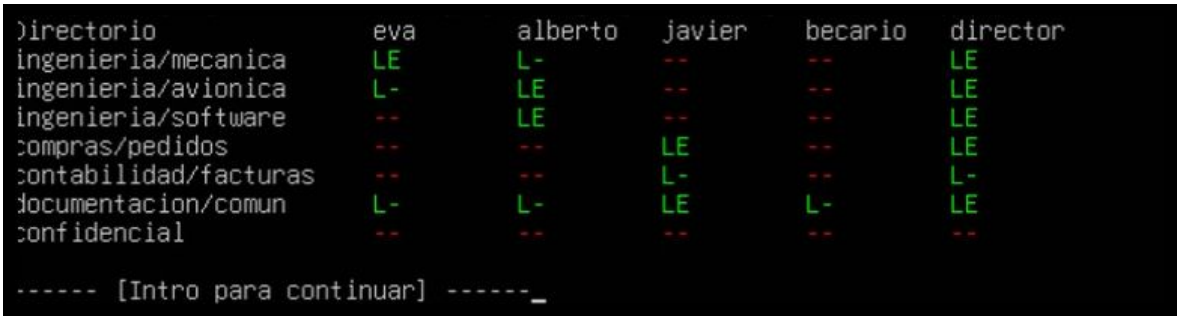


Ilustración 30. *Matriz de accesos final con las cuentas becario y director.*

2.9. Acceso remoto de los responsables por SSH

Para comprobar que cada responsable puede trabajar con su propia cuenta desde su ordenador, se ha preparado el acceso remoto al servidor. En la prueba, los usuarios `alberto` (miembro B) y `javier` (miembro C) se conectaron a `server-asio-01` desde equipos con Windows, sin necesidad de ningún servidor propio. Para ello se ha utilizado **SSH**, el protocolo que permite abrir una sesión cifrada en una

máquina remota y que ya se instaló con OpenSSH en la práctica 1. Como la máquina virtual usa red NAT, no tiene una dirección propia en el wifi; por eso VirtualBox reenvía el puerto 2222 del portátil Mac al puerto 22 de la máquina virtual, y los responsables se conectan a la dirección del Mac en la red (`10.168.3.8`) con `ssh -p 2222 usuario@10.168.3.8`.

Antes de dar acceso, se revisó desde el Terminal del Mac la regla de reenvío del puerto 2222 con `VBoxManage showvminfo`, se obtuvo la dirección del Mac en el wifi con `ipconfig getifaddr en0` y se comprobó que era posible conectarse por SSH usando esa dirección.

A continuación, desde la cuenta `director` se leyeron las contraseñas temporales del fichero de credenciales y se entregó a cada responsable solo la suya. Al conectarse por primera vez, el sistema les obligó a cambiarla, tal y como se configuró con `chage -d 0`. Una vez dentro, ambos comprobaron con `hostname` que estaban en `server-asio-01` y con `whoami` que lo hacían con su propio usuario. Además, cada uno creó un fichero en su área, `avionica` en el caso de `alberto` y `compras/pedidos` en el de `javier`, y el fichero nuevo aparece con el grupo del departamento y el signo `+`, gracias al bit setgid y a las ACL por defecto. En cambio, al intentar crear un fichero en `mecanica` el sistema les devolvió «Permission denied», lo que demuestra que los permisos se aplican igual cuando el acceso es remoto.

```
alberto@server-asio-01: $ hostname
server-asio-01
alberto@server-asio-01: $ whoami
alberto
alberto@server-asio-01: $ echo "Prueba desde el ordenador de Alberto" > avionica/remoto.txt
alberto@server-asio-01: $ ls -l avionica/
total 8
-rw-rw----+1 alberto avionica 37 Oct 19 13:03 remoto.txt
-rw-rw----+1 alberto avionica 19 Oct 19 12:09 telemetria.txt
alberto@server-asio-01: $ touch /srv/aurora6/ingenieria/mecanica/prueba.txt
touch: no se puede tocar '/srv/aurora6/ingenieria/mecanica/prueba.txt': Permission denied
alberto@server-asio-01: $ exit
logout
Connection to 10.168.3.8 closed.
```

Ilustración 31. *Usuario alberto (miembro B) conectado por SSH desde otro ordenador: escritura permitida en avionica y denegada en mecanica.*

```
javier@server-asio-01:~$ hostname
server-asio-01
javier@server-asio-01:~$ whoami
javier
javier@server-asio-01:~$ echo "Pedido desde el ordenador de Javier" > compras/pedidos/remoto.txt
javier@server-asio-01:~$ ls -l compras/pedidos
total 12
-rw-r--r-- 1 javier compras 34 Oct 19 12:10 pedido_001.txt
-rw-rw----+1 javier compras 26 Oct 19 11:35 pedido_002.txt
-rw-rw----+1 javier compras 36 Oct 19 12:50 remoto.txt
javier@server-asio-01:~$ touch /srv/aurora6/ingenieria/mecanica/prueba.txt
touch: estableciendo tiempos de '/srv/aurora6/ingenieria/mecanica/prueba.txt': Permission denied
javier@server-asio-01:~$ exit
logout
Connection to 10.168.3.8 closed.
```

Ilustración 32. *Usuario javier (miembro C) conectado por SSH desde otro ordenador: escritura permitida en compras/pedidos y denegada en mecanica.*

En estas capturas el mensaje de error es el original del sistema, ya que en este caso los comandos se escribieron directamente y no a través del script, que es el que muestra el aviso «Acceso restringido». Asimismo, en el listado de `javier` se observa que `remoto.txt`, creado desde su ordenador, tiene permisos `rw-rw----+`, por lo que el resto de Compras también podrá modificarlo.

3. Problemas encontrados y análisis de soluciones propuestas

A lo largo de la práctica han aparecido varios problemas, tanto en la configuración de los permisos como en el manejo del servidor. Para cada uno se explica por qué suponía un problema y cómo se ha resuelto, y al final se recogen todos en una tabla resumen.

El primero apareció al revisar la validación de los miembros B y C. Los ficheros `version.txt` y `pedido_001.txt` se habían creado con permisos `rw-r--r--`, de modo que el resto del departamento podía leerlos pero no modificarlos. Esto era un problema porque un departamento debe poder trabajar en equipo sobre sus documentos, y se debía a que la máscara de creación del sistema (`umask 022`) quita la escritura al grupo y a que `software` y `compras`, a diferencia de `mecanica` y `avionica`, no tenían ACL por defecto. Por lo tanto, se añadieron las ACL por defecto `d:g:software:rwx` y `d:g:compras:rwx`, y se comprobó que los ficheros nuevos ya se crean con `rw-rw----` y el signo `+`, mientras que los anteriores conservan sus permisos originales.



Ilustración 33. *Corrección: tras añadir las ACL por defecto, los ficheros nuevos son editables por el grupo.*

En segundo lugar, la primera matriz de accesos mostró «--» en todas las casillas, como si nadie tuviera acceso a nada, lo que contradecía las pruebas individuales. Era un problema porque la herramienta de comprobación daba un resultado falso, y la causa es que Ubuntu 26.04 incorpora una nueva implementación de las utilidades básicas (`coreutils` reescritas en Rust) cuyo `/usr/bin/test` no admite el operador `-a`. En consecuencia, se sustituyó por la orden interna de Bash `[ -r … ] && [ -x … ]`, que no depende de esa implementación, y la matriz pasó a reflejar correctamente la política.



Ilustración 34. *Primera matriz de accesos, incorrecta por la incompatibilidad de /usr/bin/test.*

Por otra parte, al dar permisos a Compras sobre las facturas hay que tener en cuenta que, para llegar a un directorio, se deben poder atravesar todos los anteriores. Como `contabilidad` tiene permisos `700`, la ACL de lectura de `facturas` no bastaba por sí sola, así que se añadió en `contabilidad` una ACL `g:compras:x`, que permite atravesarlo sin poder listar su contenido. De forma parecida, el becario podía entrar en `documentacion/comun` pero no leer el fichero de normas, porque la ACL de usuario se había aplicado solo al directorio y el fichero ya existía; se resolvió aplicándola de forma recursiva con `setfacl -R` y también como ACL por defecto.

Asimismo, al retirar a `eva` del grupo `sudo` la matriz de accesos volvió a salir vacía. Era lógico, ya que el script necesita privilegios de administración para ejecutar las pruebas como cada usuario y `eva` había dejado de tenerlos. Desde ese momento el script se ejecuta iniciando sesión como `director`. Del mismo modo, la versión de `sudo` de Ubuntu 26.04 guarda la autenticación por terminal, por lo que una prueba con `sudo -n`

lanzada desde otra sesión fallaba aunque `director` ya se hubiera identificado; se resolvió ejecutando esa comprobación directamente con la sesión de `director`.

En cuanto a los mensajes, el sistema informaba de los accesos denegados con textos poco claros y en inglés, como «Permission denied» o, en el caso de `sudo`, una frase humorística propia de esta versión. Esto dificultaba entender el resultado de cada prueba, de modo que el script captura los errores de permisos y los sustituye por avisos claros como «Acceso restringido: no dispones de los permisos necesarios». No obstante, cuando un usuario escribe los comandos directamente, como en el acceso remoto del apartado 2.9, sigue viendo el mensaje original del sistema.

El manejo del servidor también planteó varios problemas. Al principio el script se ejecutó desde el portátil Mac controlando la consola de VirtualBox, y algunos caracteres como `/` y `_` no llegaban correctamente a la máquina virtual por la diferencia entre la distribución de teclado del anfitrión y la de la consola. Para resolverlo se cambió la distribución con `loadkeys us` y la dirección de descarga del script se compuso con variables, guardando el carácter `/` en una variable obtenida de `$PWD` desde la raíz del sistema. Además, la consola de la máquina virtual no permite copiar y pegar, ya que el servidor no tiene entorno gráfico y el portapapeles compartido de VirtualBox solo funciona con escritorio. Por consiguiente, se decidió trabajar conectándose por SSH desde el Terminal del Mac, donde sí se puede copiar y pegar, que es además la forma habitual de administrar un servidor.

Por otra parte, al reanudar la máquina virtual desde su estado guardado el reloj del sistema estaba atrasado tres días y el servicio `systemd-timesyncd` no existe en esta versión, por lo que se ajustó la hora manualmente con `date -s` para que las fechas de los ficheros fueran correctas. Asimismo, en el servidor quedaban restos de una ejecución anterior incompleta de una versión previa del script, lo que podía mezclar resultados antiguos con los nuevos; para partir de un estado limpio se añadió la opción `limpiar`, que elimina las cuentas, los grupos y el árbol creados por la práctica, y a continuación se ejecutó todo desde cero.

Durante la ejecución, además, la petición de contraseña de `sudo` caducó por esperar demasiado y la contraseña se escribió después en la línea de comandos, donde quedó visible. Era un riesgo porque cualquiera que viera la pantalla o el historial podía leerla; por eso se ejecutó `history -c`, se limpió la pantalla y se recomendó cambiar esa contraseña. Este incidente refuerza la idea de introducir las contraseñas solo cuando el sistema las solicita.

Finalmente, al preparar el acceso remoto aparecieron dos problemas más. Por un lado, al consultar la regla de reenvío con `VBoxManage` desde una sesión del servidor como `director`, se obtuvo «command not found» y, con la ruta completa, «No such file or directory». El motivo es que `VBoxManage` pertenece a VirtualBox, que está instalado en el Mac y no dentro del servidor; por lo tanto, se cerró la sesión con `exit` y el comando se ejecutó en el Terminal del Mac con su ruta completa. Por otro lado, la regla de reenvío creada en la práctica 1 tenía la IP del anfitrión fijada en `127.0.0.1`. Esto era un problema porque `127.0.0.1` es la dirección de bucle local, que solo es accesible desde el propio Mac, así que los demás responsables no podían conectarse aunque estuvieran en el mismo wifi. Se resolvió volviendo a crear la regla con ese campo vacío, de modo que VirtualBox escucha en todas las interfaces del Mac, y se verificó conectándose por la dirección `10.168.3.8`.



Ilustración 35. *Regla de reenvío limitada a 127.0.0.1, corrección con la IP del anfitrión vacía y conexión SSH por la dirección del Mac en la red.*

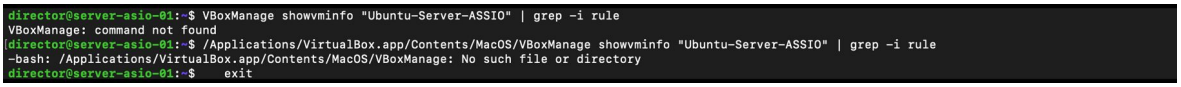


Ilustración 36. *VBoxManage ejecutado por error dentro del servidor: el comando solo existe en el Mac anfitrión.*

Problema	Causa	Solución
Ficheros nuevos sin escritura para el grupo	umask 022 y sin ACL por defecto en software y compras	ACL por defecto d:g:software:rwx y d:g:compras:rwx
Matriz de accesos vacía	/usr/bin/test de Ubuntu 26.04 no admite -a	Test interno de Bash [ ] && [ ]
Compras no llegaba a las facturas	contabilidad 700 no se podía atravesar	ACL g:compras:x en contabilidad
Becario sin lectura de las normas	ACL solo en el directorio	setfacl -R y ACL por defecto
Matriz vacía al quitar sudo a eva	El script necesita privilegios	Ejecutar el script como director
sudo -n fallaba desde otra sesión	sudo guarda la autenticación por terminal	Comprobar desde la sesión de director
Mensajes de error poco claros	Textos del sistema en inglés	Avisos «Acceso restringido» en el script
Caracteres / y _ no llegaban a la VM	Distribución de teclado distinta	loadkeys us y variables
No se podía copiar y pegar en la VM	Servidor sin entorno gráfico	Trabajar por SSH desde el Terminal
Reloj del servidor atrasado	VM reanudada desde estado guardado	Ajuste manual con date -s
Restos de una ejecución previa	Versión antigua del script a medias	Opción limpiar y ejecución desde cero
Contraseña visible en pantalla	Escrita tras caducar la petición de sudo	history -c, clear y cambio de contraseña
VBoxManage «command not found»	Se ejecutó dentro del servidor	Ejecutarlo en el Terminal del Mac
Compañeros sin acceso por SSH	Regla de reenvío con host ip 127.0.0.1	Regla con la IP del anfitrión vacía

Tabla 6. *Resumen de los problemas encontrados y de su solución.*

4. Reparto de tareas (trabajos grupales)

Eva María Ramos Juárez, como miembro A, se ha encargado de la configuración de `ingenieria/mecanica` y de su ACL de lectura para Aviónica, de su bloque de validación, de la parte del usuario `alberto` (miembro B), es decir, la configuración de `avionica` y `software`, la ACL de lectura para Mecánica y su bloque de validación, y de la preparación y ejecución del script en el servidor, así como de la memoria, de la presentación y su guion, del repositorio de GitHub y de los extras de la práctica (las mejoras del apartado 2.7: sticky bit en el intercambio, copia y restauración de permisos, plantilla /etc/skel, política de contraseñas, cuenta temporal de becario e informe de auditoría; la cuenta de dirección `director` del apartado 2.8; y el acceso remoto por SSH del apartado 2.9). Por su parte, Javier Jiahao Cheng Hu, como miembro C, ha configurado `compras`, `documentacion/comun` y la ACL de facturas, y ha realizado su bloque de validación. Finalmente, los apartados comunes, como la identificación, la estructura de directorios, los usuarios y grupos y los permisos `751`, los ha realizado también Eva María Ramos Juárez.

5. Conclusiones

En esta práctica se ha organizado el proyecto AURORA-6 en `/srv/aurora6`, se han creado los usuarios y grupos de los tres departamentos, cada responsable dispone de una cuenta propia con contraseña y de un acceso directo a su área, y se ha aplicado una política de permisos que se ha validado con cada usuario y resumido en una matriz de accesos. El resultado es que cada miembro escribe solo en su área, lee lo que necesita de otras áreas y no puede acceder a la información confidencial ni a la contabilidad sin autorización.

Además, se ha comprobado que los permisos DAC son suficientes para la mayoría de los casos, pero que se quedan cortos cuando dos departamentos necesitan permisos distintos sobre el mismo directorio. En esos casos, las ACL permiten añadir excepciones concretas sin tocar el propietario ni el grupo. Asimismo, el bit setgid y las ACL por defecto son imprescindibles para que la política se mantenga con los ficheros que se creen en el futuro, como demostró el problema de los ficheros de `software` y `compras`.

Por otra parte, las mejoras demuestran que con los mismos conceptos se pueden resolver necesidades reales: el sticky bit protege los ficheros compartidos, la copia con `getfacl` permite recuperar una configuración dañada, las ACL de usuario dan accesos puntuales sin crear grupos nuevos y separar la cuenta de dirección de los responsables aplica el mínimo privilegio también a la administración.

Para concluir, la práctica muestra la importancia de validar con las cuentas reales de los usuarios, y no como administrador, y de no dar por buena una herramienta de comprobación sin contrastarla, ya que tanto la máscara de creación como la incompatibilidad de `/usr/bin/test` solo se detectaron al revisar con atención los resultados obtenidos en el servidor.